💡 WHAT IS A TRIE?

A **prefix tree** (digital tree) stores characters as nodes. Each path from root to node represents a prefix.

> 🔑 **AHA:** Strings that share a prefix (e.g. "cat", "cater", "cats") reuse the same branch `c → a → t`. Saves space & enables instant prefix search!

📊 COMPLEXITY

Operation	Time	Space
Insert (L)	**O(L)**	O(L·N)
Search (L)	**O(L)**	O(1)
StartsWith (L)	**O(L)**	O(1)

✅ **Why O(L)?** Array indexing `children[c-'a']` is O(1), so we just walk L characters.

⚡ TRIE vs HASHMAP

`HashMap` is O(1) for **exact** lookups, but CANNOT search by prefix. Trie gives **O(L) prefix search** independent of dictionary size N.

> 🧠 **AHA:** "autocomplete", "search suggestions", "prefix match" → **Trie**

1 TRIE NODE STRUCTURE

```
public class TrieNode {
    TrieNode[] children = new TrieNode[26];  // lowercase English
    boolean isWord = false;
    // optional: String word;  // for Word Search II
    // optional: int sum;      // for MapSum
}
```

TRIE · API & DRY RUN

Insert · Search · StartsWith

STANDARD TRIE — LC 208

```
class Trie {
    private final TrieNode root = new TrieNode();

    public void insert(String word) {
        TrieNode curr = root;
        for (char c : word.toCharArray()) {
            int idx = c - 'a';
            if (curr.children[idx] == null)
                curr.children[idx] = new TrieNode();
            curr = curr.children[idx];
        }
        curr.isWord = true;
    }

    public boolean search(String word) {
        TrieNode node = find(word);
        return node != null && node.isWord;
    }

    public boolean startsWith(String prefix) {
        return find(prefix) != null;
    }

    private TrieNode find(String s) {
        TrieNode curr = root;
        for (char c : s.toCharArray()) {
            int idx = c - 'a';
            if (curr.children[idx] == null) return null;
            curr = curr.children[idx];
        }
        return curr;
    }
}
```

DRY RUN — INSERT "app"

Char	Idx	Child exists?	Action
'a'	0	❌	create → move
'p'	15	❌	create → move
'p'	15	❌	create → move
END	—	—	`curr.isWord = true` ✅

Search "app" → root → a → p → p → `isWord` == `true` ✅
Search "appl" → root → a → p → p → l → `null` ❌

> 🧠 **AHA:** Search and StartsWith share the same `find()` method; only difference is checking `isWord`.

TRIE · DECISION TREE

When to use Trie? Which variant?

DECISION TREE — PREFIX PROBLEMS

Need prefix? → Trie → Exact word? → search() ↕ Prefix? → startsWith()

Wildcard '.' search	Trie + DFS (LC 211)	Multi-word grid search	Trie + DFS + Backtracking (LC 212)
Replace with shortest root	Prefix Match (LC 648)	Prefix sum (MapSum)	Trie + DFS subtree sum
Autocomplete top 3	Store lists at nodes (LC 1268)	Suffix encoding	Reverse Trie (LC 820)

TRIGGER WORDS → VARIANT

Trigger	Variant	Key Insight
"autocomplete", "suggestions"	Standard / Top-K	Store lists at nodes
"wildcard dot"	Trie + DFS	On '.' loop all 26 children
"word boggle", "grid words"	Trie + Grid DFS	Prune invalid paths
"max XOR"	Bit Trie (2 children)	Greedy opposite bit

🗺️ TRIE · WILDCARD SEARCH

LC 211 — Add & Search Words

🔍 PROBLEM

Design a data structure that supports adding words and searching words with `.` wildcard.
`.` matches **any** single character.

> 🧠 **AHA:** When we see `.`, we must **DFS over all 26 children**. If **any** branch returns true → true.

🌲 RECURSION TREE

```
search("a.p")
   root
    └─ 'a'
        └─ '.' → try all 26 children
            ├─ 'b' → 'p' → isWord? false
            ├─ 'c' → 'p' → isWord? false
            └─ 't' → 'p' → isWord? true ✅
```

💻 CODE — LC 211

```
public boolean search(String word) {
    return searchInNode(word, 0, root);
}

private boolean searchInNode(String word, int idx, TrieNode
node) {
    if (node == null) return false;
    if (idx == word.length()) return node.isWord;

    char c = word.charAt(idx);
    if (c == '.') {
        for (TrieNode child : node.children) {
            if (child != null && searchInNode(word, idx + 1,
child))
                return true;
        }
        return false;
    } else {
        return searchInNode(word, idx + 1, node.children[c -
'a']);
    }
}
```

> ⚡ **COMPLEXITY:** O(26^L) worst-case for wildcard, but Trie prunes branches.

🗺️ TRIE · SHORTEST PREFIX

LC 648 — Replace Words

💡 IDEA

Given a dictionary of roots, replace each word in a sentence with the **shortest** matching root.

> 🧠 **AHA:** As we traverse the Trie character by character, **stop immediately** when we hit `curr.isWord == true` — that's the shortest root! Don't go deeper.

🔍 DRY RUN

roots: ["cat", "cater"]
word: "caterpillar"

```
root → c → a → t → isWord? true ✅ → return "cat"
```

✅ We stop at "cat" — we don't continue to "cater".

💻 CODE — LC 648

```
public String replaceWords(List<String> dict, String sentence)
{
    Trie trie = new Trie();
    for (String root : dict) trie.insert(root);

    String[] words = sentence.split(" ");
    StringBuilder sb = new StringBuilder();
    for (String w : words) {
        if (sb.length() > 0) sb.append(" ");
        sb.append(trie.findRoot(w));
    }
    return sb.toString();
}

// Inside Trie class:
public String findRoot(String word) {
    TrieNode curr = root;
    StringBuilder prefix = new StringBuilder();
    for (char c : word.toCharArray()) {
        int idx = c - 'a';
        if (curr.children[idx] == null || curr.isWord) break;
        prefix.append(c);
        curr = curr.children[idx];
    }
    return curr.isWord ? prefix.toString() : word;
}
```

🗺️ TRIE · GRID DFS PRUNING

LC 212 — Word Search II

🔴 WHY TRIE BEATS PLAIN DFS

Searching **M words** with plain DFS → run grid DFS **M times** → O(M · 4^L) → TLE.
Insert all words into a **single Trie**. Then one grid DFS searches **all words simultaneously**, pruning invalid branches instantly.

> 🧠 **AHA:** Store the full `word` at the leaf node. When we find it, add to result and set `node.word = null` to avoid duplicates!

🤿 DFS + BACKTRACKING

```
private void dfs(char[][] board, int r, int c,
                 TrieNode node, List<String> res) {
    char ch = board[r][c];
    if (ch == '#' || node.children[ch - 'a'] == null) return;

    node = node.children[ch - 'a'];
    if (node.word != null) {
        res.add(node.word);
        node.word = null; // 👈 prevent duplicates
    }

    board[r][c] = '#'; // mark visited

    for (int[] d : DIRS) {
        int nr = r + d[0], nc = c + d[1];
        if (nr >= 0 && nr < board.length &&
            nc >= 0 && nc < board[0].length) {
            dfs(board, nr, nc, node, res);
        }
    }

    board[r][c] = ch; // 🔄 restore
}
```

🧠 STACK FRAME INSIGHT

Why `board[r][c] = ch` ?
After exploring all directions from this cell, we **restore** the original character so other paths can use it. This is standard **backtracking**.

> ⚡ **KEY:** Without restoring, the grid would stay `'#'` and other words couldn't reuse that cell.

📊 COMPLEXITY

Time: O(R · C · 4^L) — but Trie prunes heavily in practice.
Space: O(L · N) for Trie.

🔍 NORMAL TRIE + DFS SUM

Problem: Insert (key, value). Sum all values for keys with given prefix.

> 🧠 **AHA:** Reach the prefix node, then **DFS the entire subtree** and sum all `isWord` values.

```
public int sum(String prefix) {
    TrieNode node = find(prefix);
    if (node == null) return 0;
    return dfsSum(node);
}

private int dfsSum(TrieNode node) {
    int total = node.isWord ? node.val : 0;
    for (TrieNode child : node.children) {
        if (child != null) total += dfsSum(child);
    }
    return total;
}
```

⚡ OPTIMIZED — DELTA

Optimization: Store `node.sum` = sum of all values in subtree.

When updating a key, compute **delta** = newVal - oldVal, and add delta to **every node** along the path.

> 🧠 **AHA:** Instead of DFS each time, update path nodes with delta → O(L) per insert!

```
// insert("apple", 3) → nodes: a,p,p,l,e each += 3
// insert("apple", 6) → delta = 3 → add 3 to path
// sum("app") → just return node.sum at "app"
```

🗺️ TRIE · LONGEST WORD

LC 720 — Longest Word in Dictionary

💡 THE INSIGHT

Problem: Find the longest word that can be built one character at a time, with every prefix present in the dictionary.

> 🧠 **AHA:** We need to traverse the Trie, but only move to a child **if** `child.isWord == true` . Because every prefix must be a valid word.

💻 DFS(root) — NOT DFS(root, "banana")

```
public String longestWord(String[] words) {
    Trie trie = new Trie();
    for (String w : words) trie.insert(w);

    String[] best = {""};
    dfs(trie.root, new StringBuilder(), best);
    return best[0];
}

private void dfs(TrieNode node, StringBuilder sb, String[]
best) {
    for (int i = 0; i < 26; i++) {
        TrieNode child = node.children[i];
        if (child != null && child.isWord) {
            sb.append((char)('a' + i));
            if (sb.length() > best[0].length()) {
                best[0] = sb.toString();
            }
            dfs(child, sb, best);
            sb.deleteCharAt(sb.length() - 1);
        }
    }
}
```

🔍 DRY RUN

words: ["a", "banana", "app", "appl", "ap", "apply", "apple"]

```
root
 └─ a (isWord=true) ✅
     └─ p (isWord=true) ✅
         └─ p (isWord=true) ✅
             └─ l (isWord=true) ✅
                 └─ e (isWord=true) ✅ → "apple"
```

❌ **"banana":** root → b → a → n → ... but "b" is not a word → cannot start.

> 🧠 **AHA:** We don't need to check if the current node itself is a word — we only move to children that are `isWord` .

🗺️ REVERSE TRIE · SUFFIX

LC 820 — Short Encoding of Words

💡 THE AHA — SUFFIX → PREFIX

Problem: Given words, encode them as a string with `#` separators. Minimize the length by sharing suffixes.

> 🧠 **AHA:** If word B is a **suffix** of word A, we don't need to add B separately. We can **reverse** each word → suffix becomes **prefix** → use a standard Trie!

🔍 EXAMPLE

words: ["time", "me", "emit"]
Reverse: "emit" ← "time", "em" ← "me", "time" ← "emit"

```
root
 └─ e
     └─ m
         ├─ i
         │   └─ t  (isWord=true) → "time" (reverse: "emit")
         └─ (isWord=true) → "me" (reverse: "em")
```

❌ **"me" is NOT leaf** → it's part of "time" → no extra encoding.

> 🧠 **AHA:** Only **leaf nodes** represent words that are not suffixes of any other word. We count only leafs!

💻 CODE — REVERSE TRIE

```
public int minimumLengthEncoding(String[] words) {
    TrieNode root = new TrieNode();
    // Insert reversed words
    for (String w : words) {
        TrieNode curr = root;
        for (int i = w.length() - 1; i >= 0; i--) {
            char c = w.charAt(i);
            int idx = c - 'a';
            if (curr.children[idx] == null)
                curr.children[idx] = new TrieNode();
            curr = curr.children[idx];
        }
        curr.isWord = true;
    }
    // DFS to sum lengths of leaf nodes
    return dfs(root, 0);
}

private int dfs(TrieNode node, int depth) {
    boolean isLeaf = true;
    int sum = 0;
    for (TrieNode child : node.children) {
        if (child != null) {
            isLeaf = false;
            sum += dfs(child, depth + 1);
        }
    }
    if (isLeaf) return depth + 1; // +1 for '#'
    return sum;
}
```

🗺️ BITWISE TRIE

Maximum XOR · 2 children

💡 BIT TRIE — 2 CHILDREN

Standard Trie: 26 children for lowercase letters.
Bit Trie: 2 children — bit 0 and bit 1. We insert numbers bit by bit from MSB (31) to LSB (0).

> 🧠 **AHA:** To maximize XOR, at each bit **greedily try the OPPOSITE bit** (`1 ^ bit`). If it exists, take it; otherwise take the same bit.

🔍 INSERT

```
class BitNode {
    BitNode[] child = new BitNode[2];
}

void insert(int num) {
    BitNode curr = root;
    for (int i = 30; i >= 0; i--) {
        int bit = (num >> i) & 1;
        if (curr.child[bit] == null)
            curr.child[bit] = new BitNode();
        curr = curr.child[bit];
    }
}
```

🔍 SEARCH MAX XOR

```
int maxXor = 0;
for (int num : nums) {
    BitNode curr = root;
    int xor = 0;
    for (int i = 30; i >= 0; i--) {
        int bit = (num >> i) & 1;
        int opp = 1 ^ bit;
        if (curr.child[opp] != null) {
            xor |= (1 << i);
            curr = curr.child[opp];
        } else {
            curr = curr.child[bit];
        }
    }
    maxXor = Math.max(maxXor, xor);
}
```

MASTER DECISION TREE

All Trie Variants in one view

CHOOSE YOUR TRIE

Exact prefix / autocomplete	→	Standard Trie (children[26])
Wildcard '.' search	→	Trie + DFS (LC 211)
Multi-word grid search	→	Trie + Grid DFS (LC 212)
Shortest root replacement	→	Prefix Match (LC 648)
Prefix sum (MapSum)	→	Trie + DFS subtree / Delta
Longest word (all prefixes)	→	DFS with child.isWord (LC 720)
Suffix encoding	→	Reverse Trie (LC 820)
Maximum XOR	→	Bit Trie (2 children) (LC 421)

INTERVIEWER FOLLOW-UPS

Question	Your Answer
Trie vs HashMap?	HashMap O(1) exact, but no prefix search. Trie O(L) prefix.
Array vs HashMap children?	Array faster for 26 chars. HashMap better for Unicode.
How to handle duplicates in Word Search II?	Set `node.word = null` after adding.
Why reverse for suffix encoding?	Suffix becomes prefix → standard Trie!

AHA MOMENTS · TRIE

One-liner insights for every pattern

LC 208 — Standard Trie

📌 **Trie stores prefixes** — search and startsWith share the same find() logic.

LC 211 — Wildcard

📌 **'.' means search all children** — DFS over 26 branches.

LC 648 — Replace Words

📌 **Stop at shortest prefix** — break when `curr.isWord == true`.

LC 212 — Word Search II

📌 **Trie prunes DFS** — store word at leaf, set `node.word = null` to deduplicate.

LC 677 — Map Sum

📌 **Reach prefix node → DFS subtree** — or optimize with delta on path.

LC 720 — Longest Word

📌 **Only move to child.isWord** — every prefix must be a valid word.

LC 820 — Short Encoding

📌 **Reverse suffix → prefix** — only leaf nodes count (non-suffixes).

LC 421 — Maximum XOR

📌 **Opposite bit maximizes XOR** — greedy pick `1 ^ bit`.

TRIE TEMPLATES

Copy-paste ready for interviews

STANDARD TRIE

```
class TrieNode {
    TrieNode[] children = new TrieNode[26];
    boolean isWord = false;
}
// insert, search, startsWith
```

WILDCARD DFS

```
boolean search(String w, int i, TrieNode n) {
    if (n == null) return false;
    if (i == w.length()) return n.isWord;
    char c = w.charAt(i);
    if (c == '.') {
        for (TrieNode child : n.children)
            if (child != null && search(w, i+1, child)) return true;
        return false;
    }
    return search(w, i+1, n.children[c-'a']);
}
```

GRID DFS

```
void dfs(board, r, c, node, res) {
    char ch = board[r][c];
    if (ch == '#' || node.children[ch-'a'] == null) return;
    node = node.children[ch-'a'];
    if (node.word != null) { res.add(node.word); node.word = null; }
    board[r][c] = '#';
    for (d : dirs) dfs(...);
    board[r][c] = ch;
}
```

MAP SUM (DFS)

```
int dfsSum(TrieNode node) {
    int total = node.isWord ? node.val : 0;
    for (TrieNode child : node.children)
        if (child != null) total += dfsSum(child);
    return total;
}
```

REVERSE TRIE

```
// insert reversed
for (int i = w.length()-1; i >= 0; i--) {
    char c = w.charAt(i);
    // ... standard insert
}
// count only leaf nodes
int dfs(node, depth) {
    boolean leaf = true;
    int sum = 0;
    for (child : node.children) if (child != null) { leaf=false;
sum+=dfs(child, depth+1); }
    return leaf ? depth + 1 : sum;
}
```

BIT TRIE

```
class BitNode { BitNode[] child = new BitNode[2]; }
// insert: bit = (num >> i) & 1
// max XOR: if (curr.child[1^bit] != null) take it
```

📌 TRIE CHEAT SHEET

Quick reference — 30 seconds before interview

✅ MASTERY CHECKLIST

- ☐ Implement Trie from scratch (3 min)
- ☐ Wildcard '.' search with DFS
- ☐ Shortest prefix replacement
- ☐ Search Suggestions (Top 3)
- ☐ Word Search II (Trie + Grid DFS)
- ☐ MapSum (DFS or delta)
- ☐ Longest Word (child.isWord check)
- ☐ Reverse Trie (suffix encoding)
- ☐ Bit Trie (Max XOR)

🚨 COMMON MISTAKES

- ❌ Forgetting to set `isWord = true`
- ❌ Not handling `null` children in wildcard
- ❌ Not restoring `board[r][c]` in Word Search II
- ❌ Not deduplicating with `node.word = null`
- ❌ Using reverse Trie but not counting only leaf nodes
- ❌ In Bit Trie, forgetting to OR `(1 << i)`

💡 GOLDEN RULES

Rule 1: `'.'` means loop all 26 children → DFS

Rule 2: Stop at `isWord` for shortest prefix

Rule 3: Store `word` at leaf for Word Search II

Rule 4: Reverse suffix → prefix → standard Trie

Rule 5: Bit Trie: 2 children, greedy opposite bit

Rule 6: Longest Word: only move to `child.isWord`